

DEEP-DIVE STUDY NOTES

Bidaw 학습 노트

논문 + 배경 지식 보강

Bidaw — FAST '26 paper seminar 보조자료

Bidaw: Enhancing Key-Value Caching for Interactive LLM Serving via Bidirectional Computation-
Storage Awareness

Hu et al., FAST '26

0. 배경 지식 정리

0.1 Transformer attention과 KV cache (수학)

Decoder-only LLM에서 한 토큰을 생성할 때:

$$Q = X \cdot W_Q, \quad K = X \cdot W_K, \quad V = X \cdot W_V$$
$$A = \text{softmax}(Q \cdot K^T / \sqrt{d}) \cdot V$$

- 새 토큰 1개를 생성할 때 자신의 **Q** 만 새로 계산하면 된다.
- 과거 토큰들의 **K**, **V** 는 동일한 **weight matrix**에서 나오고 **지난 라운드의 hidden state**는 변하지 않으므로 그대로 재사용 가능.
- 이 **K**, **V** 텐서들의 누적이 곧 **KV cache**.

크기 공식:

$$\text{KV size} = 2 (K + V) \cdot t (\text{tokens}) \cdot L (\text{layers}) \cdot h (\text{heads}) \cdot d (\text{head_dim}) \cdot \text{sizeof}(\text{dtype})$$

OPT-13B 예시 (FP16, L=40, h=40, d=128):

$$2 \cdot 2048 \cdot 40 \cdot 40 \cdot 128 \cdot 2 \approx 1.68 \text{ GB} / \text{user} / \text{sequence}$$

논문에서는 history 토큰만 잡아 약 **0.78 GB / user**라고 본다 (한 라운드 query+answer 단위로 끊기 때문).

0.2 MHA vs MQA vs GQA

세 가지 attention head 구조의 차이가 본 논문 § 4의 핵심 이유:

구조	Q heads	K, V heads	KV size 비율	모델 예
MHA (Multi-Head)	h개	h개 (Q와 동일)	1×	OPT, Llama-1, Bloom
GQA (Grouped)	h개	h/g개 (g=group size)	1/g×	Llama-2/3, Qwen2.5+
MQA (Multi-Query)	h개	1개	1/h×	Falcon-7B, PaLM

MHA에서 KV가 큰 이유: head별 K, V가 분리되어 있어 head 수만큼 곱해진다. GQA는 query head들이 K, V를 공유하므로 K, V 텐서 자체가 작아진다.

→ 논문 § 4 결론: **MHA에서만 storage-efficient tensor caching이 의미 있다**. GQA는 KV 자체가 이미 작아서 변환 비용이 우위를 잡아먹음.

0.3 vLLM과 PagedAttention

- vLLM (SOSP '23, Kwon et al.): LLM serving 시스템의 사실상 표준.
- PagedAttention: GPU 메모리를 작은 page (예: 16 토큰)로 나누어 OS의 가상 메모리처럼 관리. 동적 길이의 sequence를 **fragmentation 없이** 담는다.
- 본 논문은 vLLM 위에 Bidaw를 구현. vLLM의 paged 메모리 관리는 그대로 두고, **어떤 KV를 GPU에 올릴지**를 새로 결정한다.

vLLM의 한계 (논문 관점에서): - 모든 KV history를 GPU에 다 두려고 시도 → GPU 부족하면 **recompute** (vLLM baseline). - Multi-tier storage (host DRAM + SSD) 전략은 처음부터 설계에 없음.

0.4 HRRN (Highest Response Ratio Next)

1971년 Brinch Hansen이 제안한 고전 CPU 스케줄링 정책:

$$\begin{aligned}\text{Response Ratio } R &= (\text{waiting time} + \text{service time}) / \text{service time} \\ &= 1 + \text{waiting time} / \text{service time}\end{aligned}$$

- 짧은 작업이 우대받지만, 오래 기다린 긴 작업도 결국 ratio가 올라가서 **기아 방지**.
- SJF의 이점 + FCFS의 공정성을 절충.

본 논문의 **disk-HRRN**은:

$$R = 1 + \text{waiting time} / \text{KV size}$$

- **service time**을 **KV size**로 대체. KV size가 곧 capacity-layer I/O 시간에 비례한다는 가정.
- 적용 위치: preparing queue 안의 우선순위 결정 (어떤 KV를 먼저 perf layer로 promote 할지).

0.5 Belady's algorithm (1966)

- 모든 캐시 정책의 **오라클 상한**. "미래에 가장 늦게 다시 접근될 페이지를 evict".
- 현실에선 미래를 모르므로 구현 불가능 — **upper bound 측정용**으로만 사용.
- 본 논문에서 **ghost cache + Belady**의 의미: ghost cache는 **지나간** trace를 갖고 있으므로, 그 trace에 대해선 Belady가 **과거형으로** 시뮬레이션 가능. 즉 "어느 reuse-distance 구간에서 **최적이 hit를 얼마나 낼 수 있는가**"를 사후 측정.
- 그 측정값을 다음 access의 hit 확률 **추정치**로 사용한다는 게 핵심 트릭.

0.6 Two-tier KV caching의 직전 연구

시스템	venue	핵심
CachedAttention	ATC '24	Queue-enhanced LRU. Waiting queue의 KV를 미리 perf layer로 prefetch.
FlashGen	FAST '25	Inclusive caching + GPU-fit scheduling. Capacity layer에 사본을 항상 유지해 eviction 비용 0.
HCache	EuroSys '25	KV 대신 intermediate activation을 캐시 — Bidaw § 4의 storage-efficient tensor 아이디어와 가장 가까움. 하지만 scheduling/eviction은 다루지 않음.

세 시스템 모두 **compute**와 **storage**가 서로의 정보를 보지 않는다는 점에서 동일한 근본 한계를 공유한다 — 이게 논문이 짚은 root cause.

1. 문제 정의

1.1 인터랙티브 LLM serving이 새로운 워크로드인 이유

기존 LLM serving 워크로드 가정: - 한 사용자 한 질문, 짧은 query, 즉시 응답. - 라운드 수 = 1 또는 2.

인터랙티브 워크로드 (Bidaw가 다루는 것): - **한 사용자 평균 22.4 라운드** (P90 = 45). 사용자는 chat을 **세션**처럼 사용. - 각 라운드의 응답을 **읽고 생각해서** 다음 질문을 만든다 → 라운드 사이에 분 단위 간격이 자주 생김. - 평균 query 길이 36 토큰, 응답 45 토큰 — 짧음. KV의 대부분은 **과거 라운드 누적 history**.

이 차이가 KV 관리에 주는 함의: 1. KV는 한 사용자 안에서 **여러 라운드 동안 살아 있어야** 한다 (단순 single-shot이 아님). 2. 동시에 캐시되는 KV 양이 **사용자 수와 평균 세션 길이의 곱**으로 폭발한다. 3. 같은 사용자의 두 access 사이에 **분 단위**의 공백이 생기고, 그 사이 **다른 사용자의 KV**가 끼어들어 cache pollution을 만든다.

1.2 Two-tier 캐싱이 자명한 해법으로 보이는 이유

GPU memory < host memory < SSD 라는 비대칭 계층이 이미 존재. 호스트 DRAM이 80GB GPU 대비 $1.6\sim 3.2\times$ 정도 큼 ($\approx 200\text{GB}$). SSD는 TB 단위. 자연스럽게: - Tier 1 (perf): host DRAM, $\sim\mu\text{s}$ latency - Tier 2 (capacity): SSD, $\sim\text{ms}$ latency

CachedAttention/FlashGen이 이 구조를 사용. 그러나 **본 논문이 보여주는 것은**: 이 구조가 있어도 ideal과 격차가 $3.8\times / 2\times$ 발생한다는 사실 → 구조 자체보다 **운영 정책**이 문제.

1.3 이상치(ideal)의 정의

Optimal baseline = "모든 history KV가 perf layer에 들어 있다고 가정"한 가상 시스템. 즉 capacity-layer I/O를 0으로 둔 시뮬레이션.

이걸 달성할 수 있는 유일한 방법은 **충분히 큰 perf layer**다. 그러나 host DRAM은 비싸고 한정. 따라서 Bidaw의 목표는 "현재 perf layer 크기로 ideal에 **근접**"하는 것이다.

2. 세 가지 핵심 관찰

2.1 Observation 1 — KV 양이 perf layer를 초과

수치(Figure 5): - OPT-13B, 30 users/min 도착률 → 동시 캐시 KV ≈ 480 GB. - Perf layer 200 GB 대비 $2.4\times$ 초과. 최대 $3.91\times$.

함의: - Perf layer를 항상 채워두는 것이 아니라, **어떤 KV를 perf에 두고 어떤 KV를 capacity로 쫓아낼지** 매번 결정해야 한다. - Eviction이 매우 자주 일어난다 — 따라서 eviction 정책이 시스템 성능을 좌우.

2.2 Observation 2 — 약한 temporal locality

핵심 정의: weighted reuse distance - 같은 KV의 두 access 사이에, **접근된 다른 KV의 총 byte 합**. - 단순한 "몇 번째 후의 access인가"가 아니라 **byte 단위**라는 점이 중요. Cache 압력을 직접 표현.

수치(Figure 6): - 80%의 KV access가 reuse distance > 200 GB (perf layer 크기). - FIFO/LRU/queue-enhanced 모두 hit rate $\approx 20\%$. - Perf layer가 평균적으로 KV의 40.1%를 담을 수 있는데도 hit가 거의 절반 수준.

원인 (논문이 명시): - 한 사용자가 답변을 읽고 새 질문을 만드는 사이 **다른 사용자 KV**가 sandwich → 거리 폭증. - 이걸 우연이 아니라 **human-in-the-loop 워크로드의 본질적 성질**. 일반화 가능.

2.3 Observation 3 — KV loading time의 큰 분산

수치(Figure 7-8): - KV loading time의 변동계수 $CV > 90\%$ — 5초 윈도우 안에서도. - KV size 분포가 100 MB부터 800 MB+까지 — 수십 배 차이.

두 가지 원인: 1. **Bandwidth 격차**: host DRAM ≈ 30 GB/s, SSD ≈ 1.5 GB/s. 같은 KV size라도 어디 있느냐에 따라 $20\times$. 2. **KV size 분산**: 같은 layer 안에서도 사용자별 history 길이 차이.

함의: - FCFS dispatch에서 **큰 capacity-layer KV** 한 개가 머리에 박히면 그 뒤 작은 perf-layer KV들 모두 GPU idle 상태로 만든다.

3. Root cause를 한 줄로

"Compute engine과 two-tier storage가 서로 가진 정보를 흘려보내지 않는다."

이 한 문장이 두 메커니즘으로 풀린다: - **Storage** → **Compute** 정보 부재 → Mechanism 1 (I/O-aware scheduling) - **Compute** → **Storage** 정보 부재 → Mechanism 2 (Previous-answer-based eviction)
이게 곧 **bidirectional awareness**.

4. Mechanism 1 — I/O-aware Request Scheduling

4.1 왜 단순 overlap이 안 되는가

Naive 발상: GPU가 layer N을 compute할 때 layer N+1의 KV를 prefetch. 실패 이유: - Compute 한 iteration $\approx$ 수십 ms. - Capacity layer I/O $\approx$ 수백 ms. - 한 iteration overlap으로는 I/O의 **일부만** 가린다. 게다가 GPU는 **다음 토큰 1개**마다 stall.

→ 큐 자체를 재구성해야 한다.

4.2 Dual queue

- **Ready queue**: KV가 perf layer에 있는 요청. FCFS, GPU에 즉시 dispatch.
- **Preparing queue**: KV가 capacity layer에 있는 요청. 백그라운드에서 perf layer로 promote 진행.
- Ready queue의 요청만 GPU compute 자격이 있다. preparing queue는 **promote 완료 후** ready queue로 옮겨감.

핵심 디테일: - Promote 완료 후 ready queue에 삽입할 때 **원래 도착 시각 기준**으로 위치를 결정. 이게 없으면 promote 늦은 요청이 끝까지 밀려서 tail latency 폭주. - GPU memory에 안 들어가는 요청은 skip
→ ready queue 내에서도 fit 검사.

4.3 disk-HRRN

Preparing queue 내부 우선순위:

$$R = 1 + (\text{waiting time}) / (\text{KV size})$$

- 작은 KV size → 처음부터 R 높음 → 빨리 promote.
- 그러나 큰 KV도 waiting이 늘면 R이 따라 올라가 결국 promote → 기아 방지.
- "Disk"라는 이름은 **I/O 시간 $\approx$ KV size**라는 가정을 강조하기 위함.

4.4 결과 (Figure 11, 19)

- 평균 큐잉 시간 5.76s → 2.45s (-57.5%).
- Throughput 전체 +1.58× (이 한 메커니즘만으로도).

5. Mechanism 2 — Previous-answer-based Eviction

5.1 핵심 발견: 답변 길이 ↔ 다음 reuse distance

12개 시간대(8:00–19:00, hourly)에서 측정: - **Spearman ρ = 0.94 ~ 0.98** — 매우 강한 양의 상관관계.
- 정확히는 reuse distance의 하한과 이전 답변 길이가 단조 관계.

직관: 1. 답변이 길다 → 사용자가 읽는 시간이 길다. 2. 그 시간 동안 다른 사용자가 보낸 요청 수가 많다. 3. 따라서 다른 사용자 KV가 더 많이 끼어든다 → reuse distance가 최소 그만큼.

이 신호가 **bidirectional**에서 가장 새로운 부분.

5.2 그러나 거리 ≠ miss

Figure 13의 분석: - Reuse distance를 small / promising / extreme 세 영역으로 나눠 hit rate 측정. - Small (< 200 GB): 어떤 정책도 hit $\approx$ 1.0 (perf layer 안쪽). - **Promising (200 GB ~ 440 GB):** Belady(Optimal)는 거의 100%, LRU는 0–40%. - Extreme (> 440 GB): Belady도 hit $\approx$ 0.

교훈: "거리 크다 → evict" 식의 단순 규칙은 promising 구간을 학살한다. 구간별 hit 확률을 모델링해야 한다.

5.3 Ghost cache + bucketing

알고리즘 (논문 § 3.3.2):

1. Promising 영역을 **m개 fine-grained bucket**으로 분할.
2. 각 bucket의 hit rate를 ghost cache 위 Belady 시뮬레이션으로 측정. - Ghost cache = 실제 데이터 없이 metadata만 관리. 메모리 부담 없음. - Belady가 지나간 trace에 대해 가능한 이유: ghost cache가 그 trace를 다 갖고 있으므로.
3. 사용자별 과거 access 분포에서 다음 access가 각 bucket에 떨어질 확률 `prob_promising(i)` 추정.
4. Compute가 넘긴 답변 길이로 reuse distance 하한 결정 → 그보다 작은 bucket 확률을 0으로 truncate, 나머지 정규화.
5. **Equation 2:** `Overall_potential = prob_small · 1.0 + prob_extreme · 0.0 +  $\sum_i$  prob_promising(i) · hit_promising(i)`
6. Overall_potential이 가장 낮은 KV를 evict.

이름 풀이: - **prob_small**: 다음 access가 small region에 있을 확률 → hit 확률 1.0. - **prob_promising(i) · hit_promising(i)**: bucket i에 떨어졌을 때 살아남을 확률 × Optimal로 측정한 hit rate. - **prob_extreme**: extreme region 확률 → hit 확률 0.

5.4 Bidirectional이 들어오는 정확한 위치

Step 4 — 이전 답변 길이로 **lower bound truncation**. 이게 없다면 그냥 사용자 분포만으로 계산 (compute의 정보 사용 안 함). 답변 길이가 들어오면 "이번 access는 절대 small region이 아니다 / promising bucket k 이상이다"를 알 수 있어 분포를 **날카롭게** 만든다.

5.5 비용

- 결정 단계 0.35 ms (eviction trigger 시).
- Belady 시뮬레이션은 **백그라운드**에서 ghost cache가 수행.
- Eviction trigger 빈도: free perf 공간 < 5% 일 때.

5.6 결과 (Figure 18)

- vs queue-enhanced(CachedAttention): miss rate -57.6%.
- vs FIFO/LRU/LFU: miss rate -69.9%.

6. Mechanism 3 — Storage-Efficient Tensor Caching

6.1 직관적 의문

"왜 굳이 KV가 아닌 다른 tensor를 캐시?"

LLM 한 layer의 forward를 6개 tensor 단계로 분해 (논문 Figure 1과 § 4): 1. embedding (input) 2. LayerNorm 출력 3. Q, K, V projection 후 4. Attention 출력 5. FFN 입력 6. **Normalized activation** (FFN 직전, 다음 layer의 LayerNorm 입력) 7. KV tensor (별도 추출)

각 tensor의 **size**와 **재사용 가능 compute**가 다르다. **Cost efficiency** = saved compute / required space (GFLOPs / MB).

6.2 Figure 14의 결과

- Tensor 6 (정규화된 activation): **51 GFLOPs/MB** — 가장 높음.
- KV tensor: 30.5 GFLOPs/MB.
- 차이: tensor 6에 대해서는 다음 layer의 모든 연산을 다 절약, 그러면서도 정규화 후 작아짐.

→ 같은 host DRAM 공간이라면 tensor 6를 캐시하는 게 **1.67× 더 많은 compute**를 절약.

6.3 변환 비용

GPU에서 tensor 6 → KV tensor 변환은 1 step (LayerNorm + Q,K,V projection + paged store). 측정 결과: - 2048 토큰에서 < 100 ms (Figure 20(c)). - **Low-priority CUDA stream**에서 수행 → 본 추론과 별개. SM idle 시간 활용.

→ "공간 절약"의 이득이 "작은 변환 overhead"보다 훨씬 크다.

6.4 GQA에서는 무효

GQA: K, V가 query head들 사이 공유 → KV tensor 자체가 1/g 작아짐. - 변환 비용은 그대로(layer 단위 연산). - KV tensor가 작으니 cost efficiency에서 우위 사라짐. - → GQA에서는 그냥 KV tensor를 캐시.

논문 평가는 OPT, Qwen이라 MHA 위주. Llama-2 GQA에 적용하면 이 메커니즘은 비활성.

7. 구현 디테일

7.1 Mix-grained PagedAttention

vLLM의 PagedAttention은 모든 토큰을 같은 크기 블록(예: 16 토큰)으로 관리. 본 논문은: - **History 토큰** + **query 토큰**: 256 토큰 큰 블록. 길이가 **이미 결정**되어 있어 큰 블록이 효율적. - **Response 토큰**: 16 토큰 작은 블록. 길이가 **예측 불가**라 작게. - 큰 블록은 작은 블록으로 split 가능, 작은 블록은 다시 합쳐질 수 있음 → 단편화 방지.

목적: CPU↔GPU 전송 효율(큰 블록일수록 PCIe 대역폭 활용 ↑) + 메모리 단편화 감소.

7.2 Continuous batching (ORCA 류)

조기 종료된 요청을 batch에서 즉시 release → 새 요청을 빠르게 admit. vLLM이 이미 사용. Bidaw도 그대로 사용.

7.3 Inclusive caching

Eviction 시 capacity layer에 **이미 사본**이 있어, evict는 perf layer pointer 제거만 → write 트래픽 0. FlashGen에서 차용.

8. 평가 요약

8.1 환경

- A800 80GB, 200GB host DRAM, RAID-5 over 4× SATA SSD (1.5 GB/s).
- PCIe Gen 4 (~30 GB/s).
- 모델: OPT-6.7B / 13B / 30B, Qwen-7B / 14B.
- Workload: 자체 1M+ round trace, ShareGPT (Poisson 시뮬).

8.2 결과 한눈에

지표	vs CachedAttention/FlashGen
평균 latency	–최대 3.58×
Throughput (동일 latency)	+1.43–1.83×
P99 tail	–47.03%
Eviction miss rate	–69.9% (vs FIFO/LRU)
Memory sensitivity (120GB)	baseline 200GB 수준 latency

8.3 Ablation

추가하는 메커니즘	누적 throughput 향상
Vanilla (vLLM + 2-tier)	1.00×
+ I/O-aware scheduling	1.58×
+ Previous-answer eviction	1.97× (1.58 × 1.25)
+ Storage-efficient tensor	2.17× (× 1.10)

→ 세 메커니즘이 **독립적으로** 의미가 있고 곱셈으로 누적.

9. 한계와 일반화 가능성

9.1 명시적 한계

1. **Single-user KV reuse만 다름.** Cross-user KV 공유(MeanCache, ATC '24)는 직교. Bidaw + MeanCache 조합은 future work.

2. GQA에서 Mechanism 3 무효. Llama-2/3, Qwen2.5+에서는 효과 미적용.
3. ShareGPT처럼 timestamp 부재 trace에서 eviction 효과 약화 (Figure 17). Poisson 시뮬이 답변 길이와 사용자 사고 시간의 상관관계를 깨버리기 때문.
4. Single GPU node 평가. Disaggregated serving (DistServe류) / multi-node 환경 미평가.

9.2 일반화 가능 신호

- "사람이 읽고 생각하는 시간" → "다른 사용자 KV 끼어들 양". 이 인과는 chat에 본질적.
- 따라서 답변 길이 → 사용자 사고 시간 → reuse distance의 신호는 다른 인터랙티브 도메인 (음성 비서, 게임 NPC 등)에도 적용 가능.

9.3 만약 더 빠른 SSD라면?

논문이 직접 검증 (5 GB/s 시뮬). FlashGen baseline 15.18 → 20.23 user/min, Bidaw 27.81 → 30.35 user/min. 격차 줄지만 여전히 큼. Bandwidth만으로는 격차가 사라지지 않는다는 증거.

10. 자기 점검 — 5분 안에 답할 수 있어야 한다

발표 전날 한 번 입으로 답해보세요.

1. Bidaw가 받은 두 신호를 한 줄씩 설명할 수 있는가?
2. Weighted reuse distance를 정의할 수 있는가?
3. 답변 길이가 reuse distance와 양의 상관인 이유를 한 문장으로 말할 수 있는가?
4. disk-HRRN 식을 칠판에 쓸 수 있는가? Brinch Hansen의 원래 HRRN과 어떤 점만 다른가?
5. Belady 알고리즘이 ghost cache 위에서 왜 가능한가?
6. Equation 2의 세 항이 각각 무엇을 의미하는가?
7. Storage-efficient tensor가 KV tensor보다 좋은 수치적 근거는?
8. GQA에서 Mechanism 3이 작동하지 않는 이유는?
9. Ablation 결과 세 메커니즘 각각의 기여 비율은?
10. ShareGPT에서 Bidaw 효과가 약해지는 이유는?